SMART PESTICIDE SPRAYING SYSTEM

AI-Based Plant Disease Detection and Targeted Spraying Using IoT

Technical Project Documentation

A GitHub-ready reference covering project goals, proposed architecture, AI/ML techniques, hardware, software, workflow, testing, safety, and future enhancements.

Important implementation note: This document describes a proposed system architecture. Mark a component as implemented only after building and testing it. The CNN, YOLO, Raspberry Pi, Arduino, sensors, cloud dashboard, and automated pump are options to select according to the actual prototype.

Prepared for project development, future reference, GitHub documentation, and interview preparation.

1. Project Overview

The Smart Pesticide Spraying System is an AI-powered precision-agriculture project designed to detect plant diseases, estimate infection severity, and apply pesticide only to plants or areas that meet defined treatment criteria. A camera captures plant images; a trained deep-learning model classifies visible symptoms; and a controller can operate a pump and nozzle when spraying is authorized.

The intended benefits are reduced unnecessary pesticide use, less chemical wastage, improved crop-health monitoring, and more targeted agricultural operations.

2. Problem Statement

Conventional spraying often treats entire fields uniformly even when only a portion of the crop is affected. This can increase chemical consumption, costs, environmental impact, and risks to beneficial organisms. Manual inspection is also time-consuming and may miss early symptoms. The proposed system combines computer vision and controlled actuation to support more targeted decisions.

3. Objectives

- Detect supported plant diseases using AI and computer vision.
- Process plant images from a dataset and, where supported, a live camera.
- Estimate infection severity using a defined and validated method.
- Determine whether a target requires treatment according to crop-specific rules.
- Control a pump and nozzle through an embedded controller.
- Monitor environmental conditions using optional IoT sensors.
- Record predictions, sensor readings, spraying decisions, and system errors.
- Evaluate model quality and system behavior under realistic conditions.

4. Technologies and Components

AI, Machine Learning, and Computer Vision

Technology	Purpose
Python	Main programming language and integration logic.
TensorFlow / Keras	Build and train a deep-learning model.
CNN	Classify plant or leaf images into trained disease classes.
OpenCV	Camera integration and image preprocessing.
NumPy	Numerical operations and image-array handling.
Pandas	Dataset organization and analysis.
Matplotlib	Visualize training curves and model results.
YOLOv8 (optional)	Detect and locate multiple plants or affected regions when suitable annotations are available.

Hardware

Component	Purpose
Raspberry Pi	Runs the camera application and AI inference in an edge setup.
Arduino	Controls pump, relays, and other low-level hardware.
Camera / USB camera	Captures plant images.
Relay module or motor driver	Switches a compatible pump or actuator circuit.
DC pump and spray nozzle	Moves and applies liquid to the intended target.
Servo motor (optional)	Adjusts nozzle direction.
Temperature/humidity sensor	Measures environmental conditions.
Soil-moisture sensor	Measures soil moisture.
ESP32 / ESP8266 (optional)	Provides wireless data transmission.
Power supply / battery	Powers the system with suitable regulation and protection.
Mobile trolley / rover (optional)	Moves the system between plants.

The exact hardware depends on the prototype. A Raspberry Pi can run AI inference while an Arduino handles actuator control. Do not assume all listed components are required.

IoT and Monitoring

Optional components include ESP32/ESP8266 for connectivity, Firebase or another suitable storage service, and Flask for a web dashboard. A dashboard may display disease predictions, severity, environmental readings, and spraying history. These extensions are not necessary for a basic local prototype.

5. System Architecture

```
Plant / Crop
    |
    v
Camera Capture
    |
    v
Image Preprocessing (OpenCV)
    |
    v
AI Model (CNN classifier / optional YOLO detector)
    |
    v
Disease Classification
    |
    v
Severity Assessment
    |
    v
Treatment Rules + Safety Checks
    |
    v
Raspberry Pi / Arduino Controller
    |
    v
Relay or Motor Driver
    |
    v
Pump + Nozzle --> Targeted Application

Optional sensors --> IoT monitoring --> Dashboard / data log
```

The model produces a prediction; a separate decision layer applies treatment rules and safety checks before any hardware action. The pump should remain off if inference, communication, or safety checks fail.

6. Working Principle

Image acquisition: A camera captures plant or leaf images. Dataset images support model development; camera images support deployment testing.

Image preprocessing: Resize images to the model's expected dimensions, apply required normalization, and check image quality.

Disease detection: A trained CNN predicts one of the supported classes, such as healthy or a selected disease. Results depend on the training labels and image conditions.

Severity assessment: Estimate severity using labeled severity classes or a validated method such as the percentage of visibly affected leaf area. Thresholds must be crop- and disease-appropriate.

Spraying decision: Apply defined crop-specific treatment rules, confidence checks, and safety conditions. Do not use an unvalidated model score as a pesticide dose.

Actuation: If spraying is authorized, the controller operates a suitable driver and pump for a calibrated period, directing liquid toward the intended target.

Sensor monitoring: Optional temperature, humidity, and soil-moisture readings provide environmental context but do not independently confirm disease.

Logging: Record timestamps, predictions, confidence, severity, sensor readings, actuator decisions, duration, and errors.

7. AI and ML Techniques

A. Convolutional Neural Network (CNN)

A CNN learns spatial features from images. A typical architecture may include convolutional layers, ReLU activation, pooling, flattening or global average pooling, dense layers, and an output layer. Softmax is commonly used for mutually exclusive multiclass classification. The exact architecture should be selected and evaluated for the dataset.

B. Image Preprocessing and Augmentation

Resizing and normalization make images consistent with model input requirements. Training-only augmentation—such as modest rotations, flips, or brightness variation—can help the model handle natural image variation when those transformations preserve the correct label.

C. Transfer Learning (Optional)

A pretrained model such as MobileNetV2 or EfficientNet can be fine-tuned on a crop-specific dataset. Transfer learning can be useful when labeled data or compute resources are limited.

D. Object Detection (Optional)

YOLOv8 can locate objects or affected regions when trained with suitable annotations. A classifier answers what class an image belongs to; an object detector can additionally indicate where objects appear. A YOLO model is not mandatory for a single-leaf classification prototype.

E. Evaluation

- Accuracy, precision, recall, and F1-score.
- Confusion matrix and per-class results.
- Review false negatives and false positives separately.
- Test with images captured by the intended camera in real conditions.
- Use a held-out test set; avoid near-duplicate or same-plant leakage across splits.

8. Example Spraying Decision Logic

```
if disease_detected and severity == "HIGH":
    spraying_required = treatment_rules_allow()
elif disease_detected and severity == "MEDIUM":
    spraying_required = check_treatment_rules()
else:
    spraying_required = False

if not safety_checks_passed:
    spraying_required = False
```

This is illustrative control logic, not a validated treatment prescription. The actual thresholds, approved treatment, and application method must follow crop-specific expert guidance and the pesticide label.

9. Software and Hardware Integration

Raspberry Pi responsibilities

- Capture images from the camera.
- Run OpenCV preprocessing and model inference.
- Evaluate prediction and severity outputs.
- Send a defined command to the microcontroller.
- Store prediction and system events.

Arduino responsibilities

- Receive commands from the Raspberry Pi.
- Control compatible relays, drivers, pumps, and actuators.
- Enforce maximum activation time and safe default-off behavior.
- Handle communication loss and emergency-stop conditions.

ESP32 / ESP8266 responsibilities (optional)

Transmit sensor readings and operating status for remote monitoring. Choose a communication method suitable for the hardware and environment. Loss of network access must not trigger uncontrolled spraying.

10. Dataset and Model Training

Use labeled images that represent the target crop, healthy plants, and the specific diseases in scope. PlantVillage is one possible starting source; field-image datasets are also important because laboratory-style images may differ from outdoor camera images.

- Collect or download a suitable labeled dataset.
- Inspect image quality, labels, class balance, and duplicates.
- Split into training, validation, and test sets.
- Apply preprocessing and training-only augmentation.
- Train a CNN or fine-tune a pretrained model.
- Evaluate overall and per-class metrics on unseen data.
- Review errors and improve the dataset where needed.
- Save the model and class-label mapping.
- Test on images from the actual camera and environment.

11. Suggested GitHub Folder Structure

```
smart-pesticide-spraying-system/
■■■ README.md
■■■ requirements.txt
■■■ .gitignore
■■■ data/
■   ■■■ raw/
■   ■■■ processed/
■   ■■■ README.md
■■■ notebooks/
■   ■■■ model_training.ipynb
■■■ models/
■   ■■■ README.md
■■■ src/
■   ■■■ capture_image.py
■   ■■■ preprocess_image.py
■   ■■■ predict_disease.py
■   ■■■ estimate_severity.py
■   ■■■ spraying_controller.py
■   ■■■ sensor_monitor.py
■   ■■■ main.py
■■■ arduino/
■   ■■■ pump_controller.ino
■■■ dashboard/
■   ■■■ README.md
■■■ tests/
■   ■■■ test_prediction.py
■   ■■■ test_spraying_logic.py
■■■ docs/
    ■■■ system_architecture.md
    ■■■ hardware_setup.md
```

This is a proposed layout. Add files as they are implemented. Avoid committing large datasets, secrets, or model binaries unless intentionally required by the repository.

12. Environment Setup

Create a Python virtual environment:

```
python -m venv venv
```

Activate on Windows PowerShell:

```
.\venv\Scripts\Activate.ps1
```

Activate on Linux / Raspberry Pi OS:

```
source venv/bin/activate
```

Install core packages, adjusting versions to match your Python version and hardware:

```
pip install numpy pandas matplotlib opencv-python tensorflow
```

Save the tested environment dependencies:

```
pip freeze > requirements.txt
```

TensorFlow availability varies by operating system, Python version, and Raspberry Pi platform. Verify compatibility before installation.

13. Testing and Validation

Test	Purpose
Dataset validation	Check image quality, labels, duplicates, and class balance.
Model testing	Measure disease-classification performance on unseen data.
Camera testing	Check image capture under varying lighting and backgrounds.
Severity testing	Compare estimates against labeled severity examples.
Controller testing	Verify commands, timing, and safe default-off behavior.
Pump testing	Check flow, leaks, and activation duration using water first.
Integration testing	Test the full image-to-decision-to-control workflow.
Field testing	Evaluate performance on real plants and outdoor images.
Safety testing	Check emergency stop, fault handling, and unintended activation prevention.

14. Expected Outcomes

- Identify supported plant disease classes from images.
- Distinguish healthy plants from potentially affected plants.
- Estimate infection severity using a defined method.
- Trigger targeted spraying when treatment and safety rules permit.
- Record environmental readings and system activity.
- Measure whether the prototype reduces unnecessary spraying against a defined baseline.

Any claimed improvement in pesticide consumption, accuracy, or crop health must be measured experimentally rather than assumed.

15. Limitations

- Performance depends on dataset quality and similarity to field conditions.
- Different diseases and nutrient deficiencies can produce similar visible symptoms.
- Lighting, occlusion, motion, and camera quality can reduce prediction quality.
- Disease classification does not by itself determine the correct treatment.
- Severity estimation needs a separately validated method.
- Pump flow and targeting depend on hardware calibration.
- Remote monitoring depends on connectivity and infrastructure.
- Laboratory performance may not generalize to large farms.

16. Future Enhancements

- Add more crop and disease classes.
- Use YOLO-based localization for individual plants or affected regions.
- Improve severity estimation through image segmentation.
- Add GPS or plant-row localization for field tracking.

- Build a web dashboard for crop-health reports.
- Store prediction history and sensor readings in a database.
- Evaluate lightweight edge models for Raspberry Pi.
- Add confidence thresholds, manual confirmation, and fail-safe controls.
- Compare pesticide use with conventional spraying in controlled trials.

17. Safety Considerations

- Include an emergency stop and default the pump to OFF.
- Prevent activation when inference, communication, or safety checks fail.
- Test the mechanism with water before using pesticide.
- Use only approved products and follow label directions and local agricultural guidance.
- Prevent spraying toward people, animals, water bodies, or unintended areas.
- Protect electrical components from liquid exposure.
- Use appropriate protective equipment and safe handling procedures.

18. Project Status Checklist

- ■ Dataset collection and preparation.
- ■ Disease-classification model training.
- ■ Model evaluation on unseen images.
- ■ Live camera integration.
- ■ Infection-severity estimation.
- ■ Raspberry Pi / Arduino communication.
- ■ Pump and nozzle integration.
- ■ Sensor monitoring.
- ■ End-to-end testing.
- ■ Controlled field evaluation.
- ■ Optional dashboard and data logging.

Update this checklist as work is completed. Do not describe proposed or untested features as completed in a resume, report, or GitHub repository.

19. Conclusion

The Smart Pesticide Spraying System combines artificial intelligence, computer vision, embedded systems, and optional IoT monitoring to support precision agriculture. By identifying potential plant diseases and connecting validated predictions to a controlled spraying mechanism, it aims to reduce unnecessary chemical application and improve crop-health monitoring. The project offers practical experience in deep learning, image processing, hardware integration, sensor monitoring, and real-time system development.

Keywords

Smart Agriculture, Precision Farming, Plant Disease Detection, CNN, Deep Learning, TensorFlow, Keras, OpenCV, Python, Raspberry Pi, Arduino, YOLOv8, IoT, Automated Spraying, Crop Health Monitoring, Embedded Systems.